

Módulo 2 — Prompt Engineering Profesional

Capítulo 20 — Arquitecturas Basadas en Prompts

"Cuando un prompt deja de ser una instrucción aislada y pasa a formar parte de una arquitectura, comienza la verdadera ingeniería."

Objetivos de aprendizaje

- Comprender qué es una arquitectura basada en prompts y en qué se diferencia de un conjunto de prompts encadenados.
 - Diferenciar un prompt aislado de un componente arquitectónico.
 - Analizar el papel de los prompts dentro de aplicaciones empresariales.
 - Introducir los principios fundamentales de composición y reutilización que guiarán el resto del capítulo.
-

Introducción

Durante los capítulos anteriores estudiamos cómo diseñar prompts, cómo evaluarlos, cómo operarlos en producción y cómo integrarlos en conversaciones de larga duración.

Sin embargo, una aplicación moderna rara vez utiliza un único prompt.

Es habitual encontrar decenas o cientos de prompts especializados, cada uno responsable de una tarea concreta: clasificación, extracción de información, planificación, generación de respuestas, validación o coordinación de herramientas.

El desafío deja de ser escribir un buen prompt y pasa a ser **organizar un ecosistema de prompts**.

Del prompt al componente

Antes de avanzar conviene precisar el término. En este capítulo, una **arquitectura basada en prompts** es una organización explícita de prompts especializados que se relacionan mediante contratos definidos, flujos de orquestación y responsabilidades claramente delimitadas. Lo que distingue a una arquitectura de una simple colección de prompts encadenados es que cada componente tiene una interfaz conocida, puede evolucionar de forma independiente y contribuye a un objetivo mayor sin acoplar su lógica interna al resto del sistema.

En AI Engineering, un prompt debe tratarse como cualquier otro componente de software.

Posee una responsabilidad específica, una interfaz de entrada, una salida esperada y un ciclo de vida propio.



Cada prompt puede evolucionar independientemente siempre que mantenga su **contrato funcional**: el acuerdo sobre el formato de entrada que acepta y la salida que produce. Sin ese contrato, cualquier modificación en un componente se propaga en cascada hacia el resto del sistema.

Características de una arquitectura basada en prompts

Una arquitectura madura suele incorporar las siguientes propiedades:

Característica	Beneficio
Modularidad	Prompts con responsabilidades claras.
Reutilización	Un mismo prompt puede utilizarse en distintos procesos.
Bajo acoplamiento	Cambios locales con menor impacto global.
Versionado independiente	Evolución controlada de cada componente.
Observabilidad	Métricas por prompt y por flujo.

Estas propiedades no son exclusivas del AI Engineering: son principios bien establecidos en la arquitectura de software que mantienen plena vigencia cuando el componente en cuestión es

un prompt en lugar de un módulo de código.

Responsabilidades bien definidas

Un error frecuente consiste en construir un único prompt gigantesco que intenta resolver todo el problema.

Este enfoque incrementa la complejidad, dificulta las pruebas y reduce la reutilización. Además, la lógica de negocio crítica queda enterrada dentro del prompt, fuera del control directo de la aplicación. La lógica de negocio debe permanecer bajo el control de la aplicación, no dentro del modelo.

Una alternativa consiste en dividir la solución en componentes especializados:

- clasificación;
- recuperación de contexto;
- planificación;
- validación;
- generación de respuesta.

Cada componente puede evaluarse y evolucionar por separado.

Caso de estudio

Una organización desarrolla un asistente para gestionar expedientes.

Inicialmente utiliza un único prompt para interpretar la consulta, recuperar información, analizar normativa y generar la respuesta.

Con el crecimiento del proyecto aparecen dificultades para mantener el sistema: cualquier cambio en la normativa afecta al prompt completo y las pruebas deben repetirse íntegramente ante cada modificación.

El equipo decide separar esas responsabilidades en prompts especializados conectados mediante un flujo de orquestación. Cada componente expone un contrato claro: el prompt de clasificación recibe texto libre y devuelve una categoría; el de análisis normativo recibe la categoría y el expediente y devuelve una evaluación estructurada.

La solución mejora su mantenibilidad, facilita las pruebas y reduce el impacto de futuras modificaciones.

Buenas prácticas

Los principios que se listan a continuación constituyen el fundamento de toda la discusión del capítulo. Las secciones siguientes añadirán únicamente las consideraciones específicas de cada patrón.

- Diseñar prompts con una única responsabilidad.
 - Definir contratos claros entre componentes: qué entra, qué sale y en qué formato.
 - Favorecer la reutilización frente al diseño ad hoc.
 - Versionar cada prompt de forma independiente.
 - Mantener la lógica de negocio crítica bajo el control de la aplicación, no dentro del prompt.
-

Errores frecuentes

- Concentrar toda la lógica en un único prompt.
 - Acoplar prompts a un caso de uso específico que impida su reutilización.
 - No documentar las entradas y salidas de cada componente.
 - Impedir la reutilización mediante diseños excesivamente rígidos.
-

Ideas clave

- Un prompt puede convertirse en un componente arquitectónico con responsabilidad, interfaz y ciclo de vida propios.
 - La modularidad facilita la evolución controlada de la solución.
 - La arquitectura debe organizar los prompts con los mismos principios que aplica a servicios o módulos de software.
-

Transición hacia la siguiente sección

En la próxima sección estudiaremos patrones de composición de prompts y analizaremos cómo construir flujos reutilizables que integren múltiples componentes dentro de una misma solución de AI Engineering.

Módulo 2 — Prompt Engineering Profesional

Capítulo 20 — Arquitecturas Basadas en Prompts

"Una arquitectura escalable no depende de componentes más grandes. Depende de componentes mejor organizados."

Objetivos de aprendizaje

- Comprender los principios de composición de prompts.
 - Analizar estrategias para construir flujos reutilizables.
 - Diseñar arquitecturas modulares basadas en prompts especializados.
 - Introducir patrones de orquestación entre prompts.
-

Introducción

La sección anterior estableció que un prompt puede actuar como componente arquitectónico con responsabilidad única, contrato definido y ciclo de vida propio. Pocas aplicaciones empresariales resuelven un problema con un único componente.

La mayoría implementa una secuencia de prompts especializados que colaboran entre sí para alcanzar un objetivo común. En esos casos, la calidad de la solución deja de depender únicamente del diseño individual de cada prompt y pasa a depender de la forma en que estos interactúan.

Composición de prompts

La composición consiste en organizar varios prompts especializados para resolver un problema complejo mediante una secuencia de tareas más simples.

Cada componente recibe una entrada, produce una salida y delega el siguiente paso cuando corresponde.



Este enfoque favorece la separación de responsabilidades y reduce el impacto de los cambios: cuando la normativa de validación cambia, solo se modifica el componente de validación.

El contrato entre componentes

Un aspecto central de la composición es la definición del contrato entre componentes. Un contrato especifica qué información recibe un prompt, qué formato debe tener esa entrada y qué estructura tendrá la salida.

Por ejemplo, un prompt de clasificación podría recibir texto libre y comprometerse a devolver siempre un objeto con un campo `categoria` y un campo `confianza`. El componente siguiente sabe que puede consumir esa estructura sin procesamiento adicional.

Este acuerdo explícito permite sustituir o mejorar un componente sin afectar al resto del flujo, siempre que el nuevo componente respete el mismo contrato. Sin él, los componentes quedan acoplados a los detalles internos del anterior y cualquier cambio se propaga en cascada.

Beneficios de la modularidad

Una arquitectura modular ofrece ventajas tanto técnicas como operativas que resultan familiares para cualquier arquitecto de software.

Beneficio	Descripción
Reutilización	Un mismo prompt puede participar en distintos procesos.
Mantenibilidad	Los cambios afectan únicamente al componente involucrado.
Escalabilidad	Es posible incorporar nuevos módulos sin rediseñar toda la solución.
Observabilidad	Permite medir el comportamiento de cada etapa del flujo.
Pruebas independientes	Cada prompt puede validarse de forma aislada.

Patrones de composición

Existen diversas formas de organizar una arquitectura basada en prompts:

- **Pipeline secuencial:** cada prompt consume la salida del anterior.
- **Ramificación:** diferentes prompts resuelven tareas específicas en paralelo.
- **Convergencia:** múltiples resultados se integran en una respuesta unificada.
- **Orquestación condicional:** el flujo cambia según el resultado de una etapa previa.

Las secciones siguientes profundizan en estos patrones: la orquestación condicional se desarrolla en la Sección 03 a través del orquestador, y las cadenas y bifurcaciones en la Sección 04 mediante grafos de prompts.

Una consideración práctica que no debe ignorarse es el impacto acumulado sobre la latencia y el consumo de tokens. Cada componente adicional en la cadena suma tiempo de procesamiento y tokens al costo de la operación. Un diseño que encadena prompts sin una estrategia clara puede alcanzar rápidamente límites operativos que comprometan tanto el rendimiento como la economía de la solución.

Caso de estudio

Una aseguradora implementa un asistente para gestionar siniestros.

La solución se divide en cinco componentes:

1. clasificación de la consulta;
2. recuperación de información relevante;

3. validación de cobertura;
4. generación de recomendaciones;
5. construcción de la respuesta final.

Cada componente define su contrato: el de clasificación devuelve el tipo de siniestro; el de validación recibe ese tipo junto con los datos de la póliza y devuelve si la cobertura aplica y bajo qué condiciones.

Cuando cambia la normativa de cobertura, únicamente se modifica el prompt responsable de esa etapa. El resto de la arquitectura permanece inalterado porque los contratos siguen siendo válidos.

Buenas prácticas

Las consideraciones propias de la composición complementan los principios generales establecidos en la sección anterior:

- Definir contratos de entrada y salida para cada componente antes de implementarlo.
- Diseñar componentes reutilizables que no asuman contexto externo implícito.
- Evitar dependencias innecesarias entre módulos.
- Documentar cada flujo de composición.
- Evaluar el impacto acumulado sobre latencia y tokens al diseñar cadenas largas.

Errores frecuentes

- Encadenar prompts sin una estrategia de diseño que justifique cada eslabón.
- Transferir información redundante entre componentes, incrementando el consumo de tokens sin añadir valor.
- Acoplar varios procesos en un único prompt cuando podrían estar separados.
- Ignorar el impacto acumulado sobre la latencia y el costo operativo.

Ideas clave

- La composición transforma prompts individuales en arquitecturas completas capaces de resolver problemas complejos.
 - La modularidad facilita mantenimiento, pruebas y evolución independiente de cada componente.
 - Un buen diseño reduce complejidad sin perder capacidad funcional, y lo hace mediante contratos explícitos que desacoplan los componentes entre sí.
-

Transición hacia la siguiente sección

En la próxima sección estudiaremos arquitecturas jerárquicas de prompts, donde un componente coordinador distribuye responsabilidades entre prompts especializados siguiendo principios de orquestación propios del AI Engineering.

Módulo 2 — Prompt Engineering Profesional

Capítulo 20 — Arquitecturas Basadas en Prompts

"Una arquitectura jerárquica no busca concentrar decisiones. Busca distribuir responsabilidades de forma organizada."

Objetivos de aprendizaje

- Comprender el concepto de arquitecturas jerárquicas de prompts.
 - Analizar el papel de un prompt orquestador.
 - Diferenciar coordinación de ejecución.
 - Diseñar soluciones escalables mediante componentes especializados.
-

Introducción

A medida que una plataforma incorpora nuevos casos de uso, aumenta la cantidad de prompts necesarios para resolverlos.

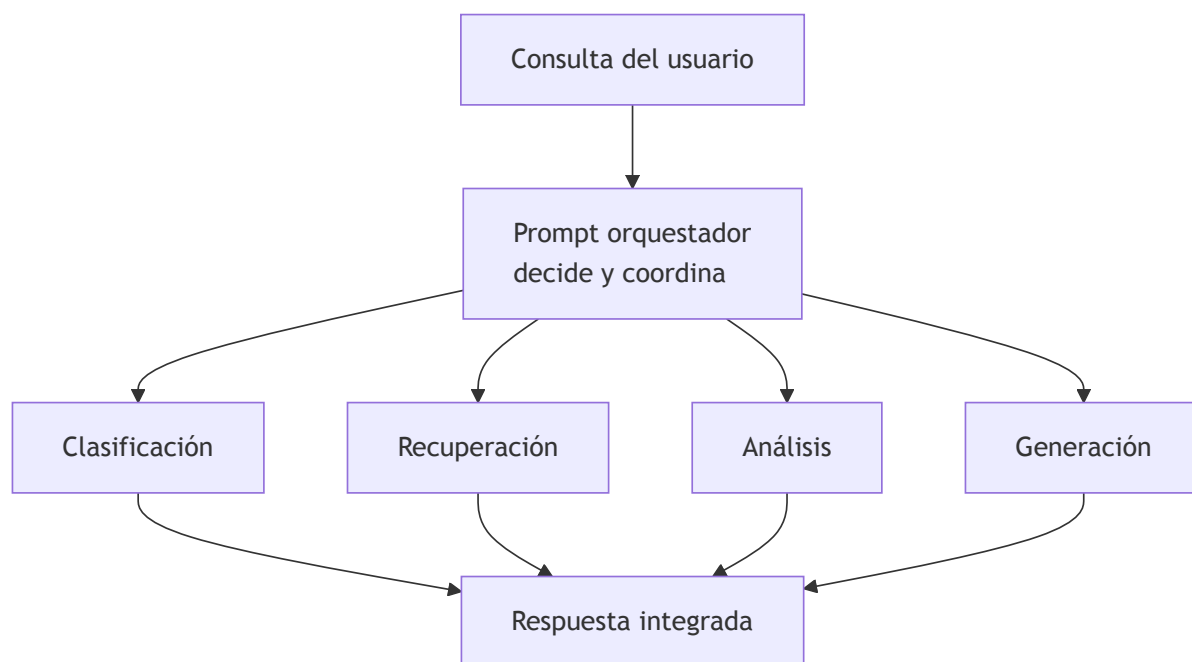
Intentar conectar todos los componentes entre sí genera dependencias difíciles de mantener. Cuando cada prompt conoce directamente a los demás, cualquier modificación puede desestabilizar el sistema completo.

Una alternativa consiste en introducir un nivel de coordinación que organice la interacción entre prompts especializados. En este enfoque aparece el **prompt orquestador**: un componente con lógica propia que decide qué otros componentes deben intervenir y en qué momento. A diferencia del pipeline secuencial, el orquestador no sigue una secuencia fija; evalúa el contexto y toma decisiones de coordinación en cada ejecución.

Arquitectura jerárquica

En una arquitectura jerárquica los prompts no colaboran de manera desordenada.

Existe un componente coordinador que interpreta la solicitud, determina la estrategia de resolución y delega cada tarea al prompt más apropiado.



La inteligencia de la solución surge de la colaboración entre componentes, no de un único prompt de gran tamaño. El orquestador no genera la respuesta final: integra los resultados parciales de los componentes especializados.

Responsabilidades del orquestador

El componente coordinador puede asumir funciones como:

Responsabilidad	Objetivo
Interpretar la intención	Comprender el objetivo principal del usuario.
Seleccionar componentes	Elegir el prompt adecuado para cada tarea.
Construir el contexto	Enviar únicamente la información necesaria.
Integrar resultados	Unificar respuestas parciales.
Gestionar errores	Definir estrategias de recuperación ante fallos.

Es importante observar que el orquestador no reemplaza a los componentes especializados. Su función consiste en coordinar su trabajo.

La gestión de errores merece atención particular. En una arquitectura jerárquica, el orquestador es el responsable de definir qué ocurre cuando un componente especializado falla: si debe reintentarse la llamada, si existe un componente de fallback que pueda asumir la tarea, o si la situación debe comunicarse al usuario. Esta responsabilidad no puede delegarse al componente fallido ni resolverse dentro del prompt; debe diseñarse explícitamente como parte de la lógica de coordinación.

Coordinación y desacoplamiento

Una arquitectura jerárquica favorece el desacoplamiento.

Cada prompt puede evolucionar de manera independiente mientras respete el contrato definido por el orquestador.

Esto permite:

- incorporar nuevos componentes sin modificar todo el sistema;
- sustituir prompts por versiones mejoradas;
- reutilizar componentes en distintos flujos;
- realizar pruebas de manera aislada.

Caso de estudio

Una organización desarrolla un asistente corporativo que responde consultas legales, financieras y de recursos humanos.

El orquestador identifica el dominio de la consulta y deriva el procesamiento al componente correspondiente. No ejecuta las consultas; evalúa la intención, selecciona el especialista adecuado y construye el contexto que ese especialista necesita.

Posteriormente integra la información obtenida y genera una respuesta consistente para el usuario.

Cuando el área financiera modifica sus políticas, únicamente evoluciona el prompt especializado de ese dominio. La arquitectura completa permanece estable porque el contrato entre el orquestador y el componente financiero no ha cambiado.

Buenas prácticas

Las consideraciones propias de la orquestación son:

- Mantener al orquestador enfocado en la coordinación; si comienza a ejecutar lógica de negocio, está asumiendo responsabilidades que no le corresponden.
- Definir contratos claros entre el orquestador y cada componente especializado.
- Minimizar el acoplamiento entre prompts especializados: deben desconocerse entre sí.
- Registrar las decisiones del orquestador para facilitar auditoría y diagnóstico.
- Diseñar explícitamente las estrategias de recuperación ante fallos de componentes.

Errores frecuentes

- Convertir al orquestador en un componente monolítico que concentra toda la lógica.
 - Duplicar responsabilidades entre prompts especializados.
 - Permitir dependencias directas entre componentes especializados, saltando al orquestador.
 - Acoplar la lógica de negocio al mecanismo de coordinación.
-

Ideas clave

- La jerarquía organiza la colaboración entre prompts mediante un componente que coordina sin ejecutar.
 - El orquestador coordina; los componentes especializados ejecutan.
 - El desacoplamiento entre componentes especializados facilita la evolución independiente de cada uno.
-

Transición hacia la siguiente sección

En la próxima sección estudiaremos arquitecturas basadas en cadenas y grafos de prompts, analizando cómo modelar flujos dinámicos que adapten su comportamiento según el contexto y los resultados obtenidos en cada etapa. Veremos también en qué se diferencia el grafo de un orquestador: mientras el orquestador es un componente con lógica propia, el grafo es una representación de la topología posible del flujo, donde las condiciones de transición guían la ejecución.

Módulo 2 — Prompt Engineering Profesional

Capítulo 20 — Arquitecturas Basadas en Prompts

"Las arquitecturas más flexibles no siguen siempre el mismo camino. Deciden el camino mientras avanzan."

Objetivos de aprendizaje

- Comprender las diferencias entre cadenas y grafos de prompts.
 - Analizar flujos secuenciales y dinámicos.
 - Diseñar arquitecturas adaptativas basadas en decisiones.
 - Introducir criterios para modelar rutas de ejecución.
-

Introducción

Las primeras aplicaciones construidas con Large Language Models (LLM) resolvían los problemas mediante secuencias lineales de prompts.

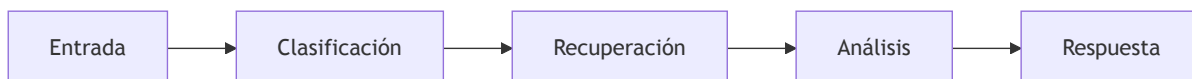
Con el crecimiento de la complejidad aparecieron escenarios donde el siguiente paso dependía del resultado obtenido en el anterior.

En esos casos, una cadena fija deja de ser suficiente y la arquitectura comienza a comportarse como un grafo de decisiones. Vale la pena precisar la distinción con el orquestador estudiado en la sección anterior: el **orquestador** es un componente con lógica propia que coordina; el **grafo de prompts** es una topología que describe la estructura posible del flujo, con nodos que representan componentes y transiciones etiquetadas que expresan las condiciones bajo las cuales el flujo avanza de un nodo al siguiente. La diferencia es entre un componente y una representación arquitectónica.

Cadenas de prompts

Una cadena representa una secuencia predefinida.

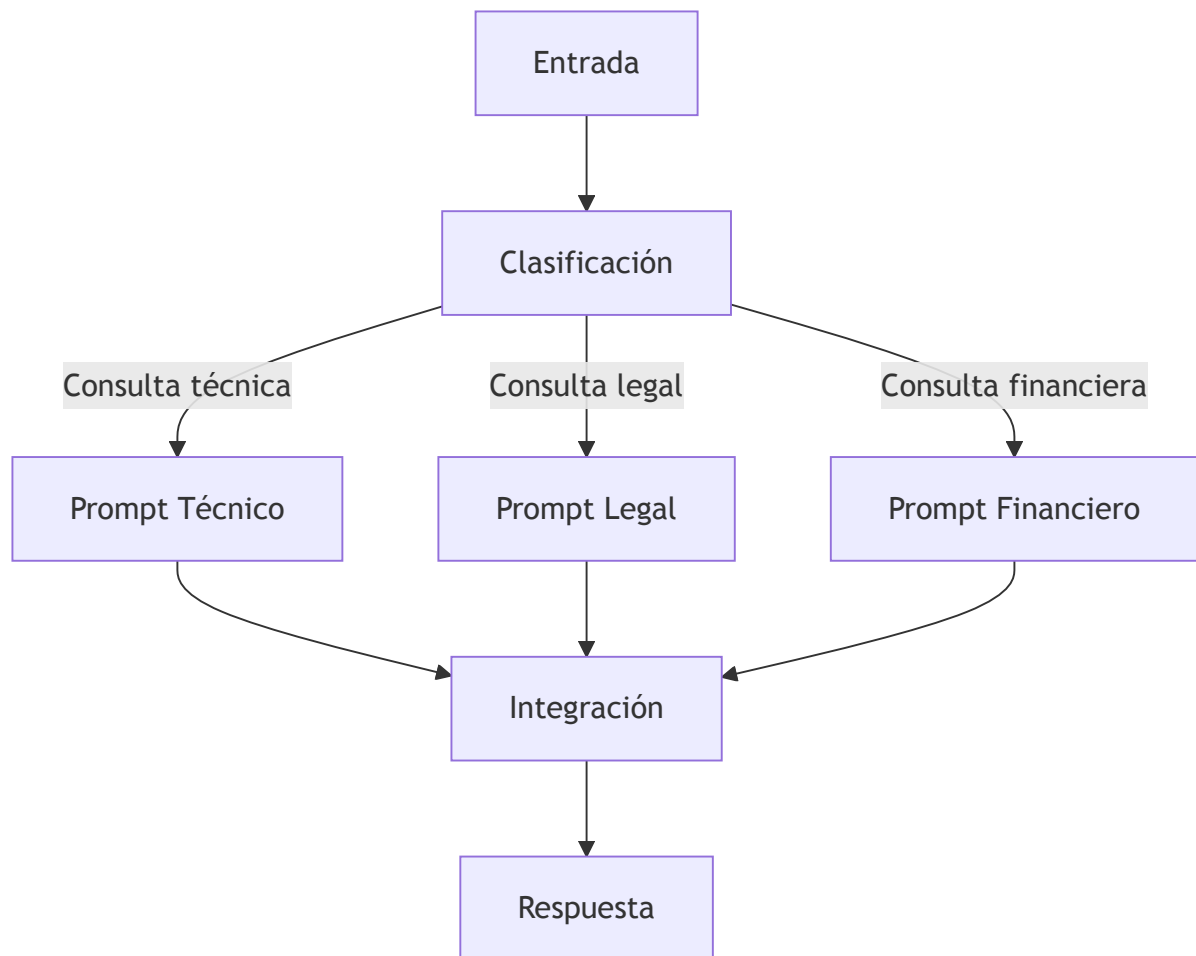
Cada componente recibe la salida del anterior y ejecuta una única responsabilidad.



Las cadenas resultan apropiadas cuando el proceso es estable y todas las ejecuciones siguen el mismo recorrido.

Grafos de prompts

En un grafo, el flujo puede bifurcarse según el contexto, el resultado de una validación o la información recuperada. A diferencia del diagrama del orquestador, las transiciones entre nodos llevan condiciones explícitas que determinan qué camino seguirá cada ejecución.



Cada nodo representa un componente especializado y cada transición responde a una condición arquitectónica explícita. No hay un componente central que decida: la decisión está codificada en las reglas de transición del grafo.

¿Cuándo utilizar cada enfoque?

Arquitectura	Escenario recomendado
Cadena	Procesos repetitivos y lineales.
Grafo	Procesos con múltiples alternativas o condiciones dinámicas.
Híbrida	Soluciones empresariales complejas.

En la práctica, la mayoría de las plataformas modernas combinan ambos modelos. Una arquitectura puede usar cadenas para las etapas estables y grafos para las secciones del flujo

que requieren decisiones condicionales.

Decisiones dinámicas

Una arquitectura basada en grafos permite decidir en tiempo de ejecución:

- qué prompts ejecutar;
- cuáles omitir;
- qué herramientas invocar;
- cuándo finalizar el proceso;
- cuándo solicitar información adicional.

La aplicación deja de ejecutar una secuencia rígida y pasa a construir el recorrido más adecuado para cada situación.

Esta flexibilidad tiene un costo que debe considerarse en el diseño: cada nodo activado implica una llamada al modelo, con el correspondiente impacto sobre la latencia y el consumo de tokens. En un grafo con múltiples bifurcaciones, el número de nodos que se activan varía según el caso. Diseñar el grafo con criterio —activando solo los nodos necesarios y manteniendo reglas de transición explícitas y verificables— es parte del trabajo del arquitecto.

Caso de estudio

Un sistema de soporte técnico multinivel recibe incidencias de distinta naturaleza.

El primer componente clasifica la incidencia según su tipo: software, hardware o conectividad.

A partir de esa clasificación, el flujo deriva hacia el prompt especializado correspondiente, que a su vez puede recuperar documentación técnica específica mediante Retrieval-Augmented Generation (RAG) o escalar a un nivel de soporte más avanzado si la incidencia supera su capacidad de resolución.

El recorrido cambia para cada incidencia sin modificar la arquitectura general. El grafo define los caminos posibles; la condición de clasificación decide cuál recorrer.

Buenas prácticas

- Diseñar nodos con responsabilidades bien definidas.
 - Mantener reglas explícitas y verificables para las transiciones; la lógica de decisión no debe ocultarse dentro de los prompts.
 - Evitar grafos excesivamente complejos: si el grafo se vuelve difícil de leer, probablemente deba revisarse la estructura.
 - Registrar el camino seguido durante cada ejecución para facilitar diagnóstico y auditoría.
 - Considerar el impacto sobre latencia y tokens al diseñar el número de nodos posibles en cada ruta.
-

Errores frecuentes

- Modelar todos los procesos como cadenas lineales cuando el problema requiere bifurcaciones.
 - Incorporar bifurcaciones innecesarias que complican el grafo sin añadir valor.
 - Ocultar la lógica de decisión dentro de los prompts en lugar de expresarla como condiciones de transición.
 - No registrar las rutas recorridas, lo que impide diagnosticar comportamientos inesperados.
-

Ideas clave

- Las cadenas simplifican procesos lineales y estables; los grafos permiten adaptar el flujo a cada situación particular.
 - La diferencia entre orquestador y grafo es la diferencia entre un componente con lógica y una topología con condiciones: conceptos complementarios, no equivalentes.
 - La arquitectura debe equilibrar simplicidad, flexibilidad y mantenibilidad, y considerar el costo operativo de cada ruta posible.
-

Transición hacia la siguiente sección

En la próxima sección estudiaremos cómo integrar arquitecturas basadas en prompts con RAG, Tool Calling y agentes, mostrando cómo estos componentes colaboran dentro de una plataforma moderna de AI Engineering.

Módulo 2 — Prompt Engineering Profesional

Capítulo 20 — Arquitecturas Basadas en Prompts

"Una arquitectura moderna no elige entre prompts, RAG o herramientas. Los integra para resolver problemas reales."

Objetivos de aprendizaje

- Comprender cómo se integran los prompts con otras capacidades de IA.
 - Analizar el papel de RAG, Tool Calling y agentes dentro de una arquitectura.
 - Diferenciar responsabilidades entre componentes.
 - Diseñar soluciones desacopladas y extensibles.
-

Introducción

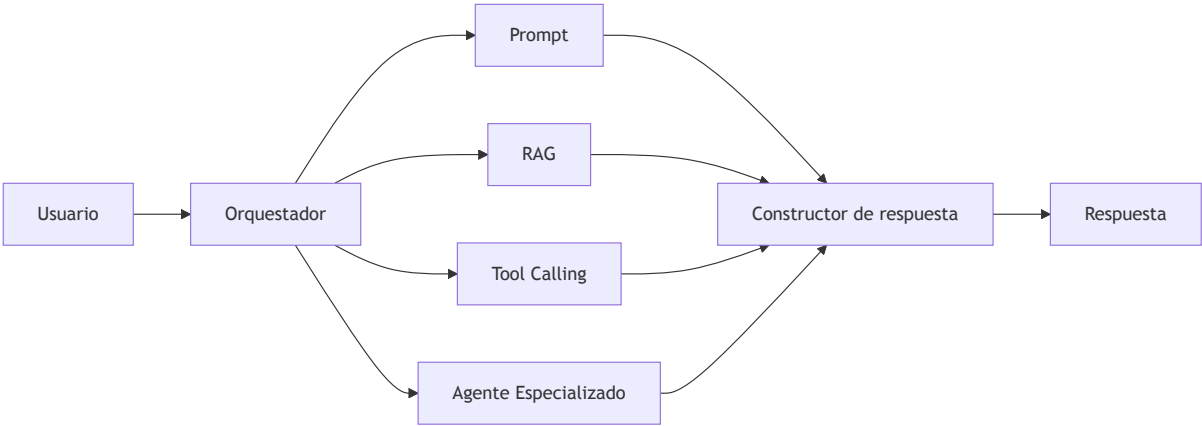
Las arquitecturas basadas en prompts constituyen el núcleo lógico de muchas aplicaciones modernas, pero rara vez operan de forma aislada.

Una solución empresarial suele combinar múltiples capacidades:

- prompts especializados;
- recuperación de información mediante Retrieval-Augmented Generation (RAG);
- ejecución de herramientas mediante Tool Calling;
- integración con APIs;
- agentes especializados;
- motores de workflow.

El desafío consiste en coordinar estos componentes sin convertir la solución en un sistema monolítico.

Arquitectura integrada



En esta arquitectura, el prompt deja de ser el único protagonista y pasa a colaborar con otros componentes especializados. El orquestador decide qué componentes intervienen; el constructor de respuesta integra sus salidas en un mensaje coherente para el usuario.

Responsabilidades de cada componente

Componente	Responsabilidad
Prompt	Interpretar instrucciones y generar razonamiento.
RAG	Recuperar conocimiento externo actualizado.
Tool Calling	Ejecutar acciones verificables sobre sistemas externos.
Agente Especializado	Coordinar tareas complejas con cierto grado de autonomía.
Orquestador	Decidir el flujo y administrar el estado.
Constructor de respuesta	Sintetizar las salidas de los distintos componentes en una respuesta coherente.

Cada componente tiene una responsabilidad única y una interfaz definida. Esta separación favorece la mantenibilidad y la evolución independiente de cada elemento.

Vale la pena precisar tres términos que se usan en la tabla. **RAG** (Retrieval-Augmented Generation) es el patrón que combina recuperación de información con generación de texto:

el sistema recupera documentos relevantes y los incorpora al contexto del modelo antes de generar la respuesta. **Tool Calling** es la capacidad del modelo de invocar funciones o APIs externas para ejecutar acciones que van más allá de la generación de texto: consultar una base de datos, leer el estado de un sistema o escribir en un registro. Un **Agente Especializado** es un componente que puede tomar decisiones y ejecutar acciones de manera autónoma para alcanzar un objetivo, gestionando su propio ciclo de razonamiento y acción. Estos tres patrones se desarrollarán en profundidad en módulos posteriores; aquí se presentan en su rol dentro de una arquitectura integrada.

El constructor de respuesta

El constructor de respuesta es el componente que recibe las salidas del prompt, el RAG, el Tool Calling y el agente especializado, y las sintetiza en una respuesta única y coherente para el usuario.

Su función no es generar razonamiento nuevo sino integrar información ya procesada: eliminar redundancias entre las distintas salidas, resolver posibles contradicciones y construir una respuesta que respete el formato y el tono esperados.

Sin un constructor de respuesta explícito, la integración de múltiples fuentes queda sin responsable definido, lo que suele derivar en respuestas inconsistentes o que exponen al usuario la estructura interna del sistema.

Cuándo usar RAG versus contexto estático

Una decisión frecuente en el diseño de arquitecturas integradas es cuándo incorporar información mediante RAG y cuándo incluirla directamente en el contexto del prompt.

RAG resulta preferible cuando el conocimiento es dinámico, se actualiza con frecuencia, tiene un volumen que supera el espacio disponible en el contexto, o solo es relevante para un subconjunto de las consultas. Incluir la información directamente en el contexto es apropiado cuando es estática, breve y siempre relevante para el componente que la recibe.

Esta decisión tiene consecuencias sobre la latencia, el costo y la calidad de la respuesta. Incorporar RAG añade una etapa de recuperación al flujo; incluir información en el contexto

consume tokens en cada llamada aunque esa información no sea necesaria.

Principios de integración

Al diseñar una arquitectura integrada conviene respetar algunos principios:

- cada componente debe tener una responsabilidad única;
- la información debe circular mediante contratos claros;
- el contexto debe construirse dinámicamente, enviando solo lo necesario a cada componente;
- las decisiones deben ser observables y auditables;
- la lógica de negocio debe permanecer bajo el control de la aplicación cuando sea posible.

Estos principios permiten escalar la solución sin incrementar innecesariamente la complejidad.

Caso de estudio

Una empresa implementa un asistente para soporte técnico.

El flujo de resolución incluye:

1. clasificar la consulta mediante un prompt;
2. recuperar documentación técnica utilizando RAG;
3. consultar el inventario de equipos mediante una API a través de Tool Calling;
4. ejecutar un diagnóstico con una herramienta especializada;
5. sintetizar toda la información en una respuesta personalizada mediante el constructor de respuesta.

Cada componente cumple una función concreta. El orquestador decide cuándo interviene cada uno según el tipo de consulta. El constructor de respuesta integra los resultados en un mensaje que el usuario puede leer sin necesidad de conocer la arquitectura subyacente.

Buenas prácticas

- Diseñar componentes independientes con responsabilidades claras.
 - Utilizar RAG para conocimiento dinámico o voluminoso; reservar el contexto estático para información breve y siempre relevante.
 - Reservar Tool Calling para acciones verificables sobre sistemas externos.
 - Incluir al constructor de respuesta en la tabla de responsabilidades y en el diseño explícito del sistema.
 - Delegar la coordinación en el orquestador; delegar la síntesis en el constructor de respuesta.
 - Medir el desempeño de cada etapa del flujo.
-

Errores frecuentes

- Resolver todos los problemas con un único prompt, ignorando las capacidades complementarias disponibles.
 - Duplicar información entre RAG y contexto, incrementando el costo sin añadir valor.
 - Mezclar razonamiento con lógica de integración dentro del mismo componente.
 - Acoplar herramientas directamente al modelo sin pasar por la capa de orquestación.
 - Omitir el constructor de respuesta del diseño, dejando la integración de resultados sin responsable definido.
-

Ideas clave

- Las arquitecturas modernas integran múltiples capacidades: prompts, RAG, Tool Calling, agentes y un constructor de respuesta.
 - Los prompts constituyen un componente más del ecosistema, no el centro exclusivo de la solución.
 - La separación de responsabilidades entre todos los componentes, incluido el constructor de respuesta, es lo que permite escalar la solución con calidad.
-

Transición hacia la siguiente sección

En la próxima sección analizaremos patrones arquitectónicos reutilizables para aplicaciones basadas en Large Language Models (LLM) y construiremos un catálogo de referencia que servirá de base para los módulos de RAG, Agentes y AI Engineering.

Módulo 2 — Prompt Engineering Profesional

Capítulo 20 — Arquitecturas Basadas en Prompts

"Las arquitecturas maduras no nacen de la improvisación. Surgen de patrones que demostraron ser útiles una y otra vez."

Objetivos de aprendizaje

- Comprender el concepto de patrón arquitectónico aplicado a aplicaciones con Large Language Models (LLM).
 - Identificar patrones reutilizables en soluciones basadas en prompts.
 - Analizar criterios para seleccionar un patrón adecuado.
 - Preparar las bases para arquitecturas de RAG y Agentes.
-

Introducción

En ingeniería de software, un patrón arquitectónico representa una solución reutilizable para un problema recurrente.

No constituye una implementación específica, sino una guía que resume experiencias acumuladas y buenas prácticas.

Las aplicaciones basadas en LLM presentan desafíos similares. Con el tiempo han surgido patrones que permiten organizar prompts, herramientas, memoria y flujos conversacionales de manera consistente.

Comprender estos patrones facilita el diseño de soluciones más robustas y reduce la necesidad de reinventar la arquitectura en cada proyecto.

¿Qué es un patrón arquitectónico?

Un patrón arquitectónico describe cómo se relacionan distintos componentes para resolver un tipo de problema.

No impone tecnologías concretas ni modelos específicos.

Su propósito consiste en ofrecer una estructura que pueda adaptarse a distintos contextos.



El patrón actúa como una referencia que guía las decisiones de diseño, no como una receta que debe seguirse al pie de la letra.

Patrones frecuentes en aplicaciones con LLM

A continuación se presentan los patrones más utilizados, incluyendo los desarrollados en las secciones anteriores del capítulo.

Patrón	Objetivo principal
Pipeline	Resolver tareas mediante un flujo lineal y predecible.
Router	Derivar solicitudes al componente más adecuado según el tipo de consulta.
Orquestador	Coordinar múltiples componentes especializados con lógica de decisión centralizada.
Retrieval-Augmented	Incorporar conocimiento externo mediante RAG para enriquecer el contexto del modelo.
Workflow	Modelar procesos con estados persistentes y reglas de negocio explícitas que pueden abarcar múltiples etapas o sesiones.
Multiagente	Distribuir responsabilidades entre agentes con capacidad autónoma de decisión y acción, cada uno especializado en un dominio.

Los patrones Pipeline, Router, Orquestador y Retrieval-Augmented se han desarrollado en las secciones anteriores. Workflow y Multiagente merecen una aclaración.

El patrón **Workflow** se distingue de un pipeline o un orquestador por la presencia de estado persistente entre etapas: el proceso puede detenerse, reanudarse en otro momento y seguir reglas de negocio explícitas que determinan las transiciones. Un trámite administrativo que requiere aprobaciones en distintos momentos y guarda el estado entre sesiones es un ejemplo típico.

El patrón **Multiagente** se diferencia del Orquestador en el grado de autonomía de los componentes: mientras que en una arquitectura orquestada los componentes especializados son prompts que esperan instrucciones del orquestador, en una arquitectura multiagente los agentes tienen capacidad de tomar decisiones propias, observar su entorno y actuar sin instrucción explícita en cada paso. Este patrón se desarrollará en profundidad en el módulo dedicado a agentes.

¿Cómo seleccionar un patrón?

La elección no depende únicamente de la tecnología disponible.

Conviene analizar aspectos como:

- naturaleza del problema;
- cantidad de componentes involucrados;
- necesidad de escalabilidad;
- requisitos de mantenimiento;
- volumen esperado de usuarios;
- complejidad del flujo de negocio.

Para orientar la decisión, algunos indicadores concretos: si el proceso es siempre el mismo para todas las consultas, un Pipeline es suficiente. Si distintos tipos de consulta requieren tratamientos distintos pero el flujo de cada uno es lineal, un Router agrega la derivación necesaria. Si el proceso requiere coordinar múltiples componentes con lógica de decisión entre pasos, un Orquestador aporta esa capa. Si el proceso incluye estados persistentes y reglas de negocio que trascienden una sola sesión, el patrón Workflow es el apropiado. Si se necesita que múltiples componentes operen con autonomía en paralelo, la arquitectura Multiagente es el nivel siguiente.

En muchos casos, la mejor solución consiste en combinar varios patrones dentro de una misma arquitectura.

Caso de estudio

Una empresa de servicios financieros desarrolla una plataforma de onboarding para nuevos clientes.

Durante la primera etapa implementa un **Pipeline** para procesar los datos del formulario de alta: validación, verificación de identidad y apertura de cuenta.

Cuando incorpora múltiples tipos de productos, agrega un **Router** que deriva cada solicitud al flujo correspondiente según el tipo de cuenta solicitada.

Al necesitar consultar normativa regulatoria actualizada, incorpora un componente **Retrieval-Augmented** que recupera los requisitos vigentes para cada producto.

Finalmente, para gestionar los procesos de aprobación que requieren intervención humana en distintos momentos y pueden extenderse varios días, agrega un **Workflow** que mantiene el estado del trámite entre sesiones.

La arquitectura evoluciona sin reemplazar los componentes existentes. Cada nuevo patrón amplía las capacidades del sistema respetando el diseño previo.

Buenas prácticas

- Seleccionar patrones en función del problema y no de la tecnología o la novedad.
 - Favorecer la composición de patrones sencillos antes de recurrir a uno más complejo.
 - Documentar las decisiones arquitectónicas y el razonamiento detrás de cada patrón elegido.
 - Mantener desacopladas las responsabilidades entre patrones combinados.
-

Errores frecuentes

- Elegir patrones por moda o por familiaridad tecnológica en lugar de por adecuación al problema.
- Implementar arquitecturas excesivamente complejas desde el inicio, cuando un pipeline simple resolvería el problema.

- Confundir un patrón con una herramienta específica: el patrón es una guía estructural, no una biblioteca.
 - Reemplazar una arquitectura estable sin una necesidad justificada.
-

Ideas clave

- Los patrones arquitectónicos representan conocimiento reutilizable acumulado en la práctica del AI Engineering.
 - No existe un patrón universal: la elección depende de las características del problema, no de la preferencia tecnológica.
 - Una arquitectura madura suele combinar varios patrones complementarios, cada uno resolviendo el aspecto del problema para el que fue diseñado.
-

Transición hacia la siguiente sección

En la próxima sección construiremos un catálogo de referencia que reunirá las arquitecturas estudiadas en el capítulo y mostrará cómo evolucionan desde soluciones simples hasta plataformas modernas de AI Engineering.

Módulo 2 — Prompt Engineering Profesional

Capítulo 20 — Arquitecturas Basadas en Prompts

"Una buena arquitectura no intenta prever todas las soluciones posibles. Intenta facilitar su evolución."

Objetivos de aprendizaje

- Construir un mapa de referencia de arquitecturas basadas en prompts.
 - Comprender la evolución desde soluciones simples hasta plataformas complejas.
 - Identificar cuándo aumentar el nivel de sofisticación arquitectónica.
 - Integrar los conceptos desarrollados a lo largo del capítulo.
-

Introducción

Durante este capítulo analizamos cómo los prompts dejan de ser instrucciones aisladas para convertirse en componentes arquitectónicos.

Estudiamos modularidad, composición, orquestación, cadenas, grafos e integración con RAG, Tool Calling y agentes.

En esta sección reuniremos todos estos conceptos en un catálogo de referencia que servirá como guía para seleccionar la arquitectura más adecuada según la complejidad del problema.

Evolución arquitectónica

Las arquitecturas basadas en prompts suelen evolucionar de manera incremental. Cada nivel incorpora nuevas capacidades sin reemplazar necesariamente las del nivel anterior.



El diagrama refleja que RAG, herramientas y agentes no son etapas secuenciales independientes sino capacidades que pueden incorporarse en conjunto al alcanzar el nivel de integración avanzada. Su separación o combinación depende de las necesidades del proyecto, no de una secuencia tecnológica obligatoria.

Catálogo de referencia

Nivel	Características	Escenarios recomendados
Prompt único	Una única responsabilidad, sin flujo entre componentes.	Automatizaciones simples y tareas puntuales.
Pipeline	Flujo lineal de procesamiento con etapas predecibles.	Procesos repetitivos y sin bifurcaciones.
Modular	Componentes especializados con contratos definidos.	Aplicaciones empresariales con múltiples funcionalidades.
Orquestado	Coordinación dinámica mediante un componente con lógica de decisión.	Sistemas complejos con múltiples dominios o tipos de consulta.
Integrado	RAG, Tool Calling, agentes y constructor de respuesta coordinados.	Plataformas corporativas con necesidad de conocimiento externo y acciones verificables.
Multiagente	Agentes con autonomía de decisión especializados por dominio.	Ecosistemas de IA avanzados donde múltiples procesos operan en paralelo con independencia.

La diferencia entre el nivel Orquestado y el nivel Multiagente merece precisión: en el primero, el orquestador coordina prompts especializados que responden a instrucciones; en el segundo, los agentes tienen capacidad autónoma de razonar, observar su entorno y decidir sin instrucción explícita en cada paso. Esta diferencia en el grado de autonomía es la que define el salto entre ambos niveles.

La transición entre niveles responde al crecimiento de los requisitos del negocio y no a una preferencia tecnológica.

¿Cuándo evolucionar la arquitectura?

No existe una regla universal. Sin embargo, algunos indicadores habituales son:

- aumento de funcionalidades que superan la capacidad del nivel actual;
- crecimiento del número de prompts hasta el punto en que la gestión manual se vuelve inviable;
- incorporación de nuevas fuentes de información que requieren recuperación dinámica;
- necesidad de reutilización entre proyectos o equipos;
- incremento del volumen de usuarios que exige mayor escalabilidad;
- mayores exigencias de gobernanza y auditoría.

La evolución debe producirse cuando la arquitectura actual deja de satisfacer los objetivos del sistema, no antes. La complejidad prematura es uno de los errores más frecuentes en el diseño de plataformas de AI Engineering.

Caso de estudio

Una empresa inicia un proyecto con un único asistente para responder consultas frecuentes.

Con el tiempo incorpora documentación técnica mediante Retrieval-Augmented Generation (RAG), integra el sistema ERP utilizando Tool Calling y agrega agentes especializados para distintas áreas del negocio.

La plataforma evoluciona gradualmente sin perder coherencia porque cada etapa se apoya sobre principios arquitectónicos previamente establecidos: contratos entre componentes, responsabilidades bien delimitadas y lógica de negocio bajo el control de la aplicación.

Ideas clave

- Las arquitecturas evolucionan junto con las necesidades del negocio; la complejidad se introduce cuando los requisitos lo justifican.
- La modularidad facilita el crecimiento controlado: un nivel puede incorporarse sin reemplazar el anterior.

- El objetivo final no es construir más componentes, sino resolver problemas con mayor calidad y menor complejidad operativa.
 - La distinción entre orquestación y multiagente radica en el grado de autonomía de los componentes, no en la cantidad de nodos del sistema.
-

Transición hacia el siguiente capítulo

En el próximo capítulo comenzaremos la parte práctica del módulo mediante una serie de laboratorios donde aplicaremos todos los conceptos estudiados sobre Prompt Engineering, Ingeniería Conversacional y Arquitecturas Basadas en Prompts para resolver problemas reales de AI Engineering.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."